

NEULASER

(v2)

Kexin Qiao, Heng Chang, Yaoling Ding, and Liehuang Zhu
Beijing Institute of Technology, Beijing 100081, China
{qiao.kexin, heng_chang, dyl19, liehuangz}@bit.edu.cn

June 30, 2026

Abstract

This document presents the NEULASER hash function family, which has a digest size of 512, 768 or 1024 bits and support extensible output length. It supports a security level that is no lower than that achieved when considering general attack methods. Firstly, a hash construction based on pseudorandom number generator (PRNG) is proposed. Each padded message block is interpreted as the seed/key input of one PRNG invocation, while the chaining value obtained from the previous output stream becomes the initial vector for the next block. The final digest is derived from a separate output portion of the last generated stream. The security of the hash function is reduced to that of the underlying PRNG. Then, a specific PRNG primitive is detailed, which consists of $\mu \in \{3, 4, 5\}$ coupled 16-stage nonlinear feedback shift-register (NLSR) modules. The primitive is designed to be efficient and secure against known attacks. The hash function is then instantiated with the proposed PRNG primitive. The classical security is analyzed in the ideal PRNG model. Its quantum security is analyzed by quantum lifting from the classical security. Finally, implementation notes and performance benchmarks are provided.

Contents

1	Introduction	3
2	PRNG-Based Construction	3
2.1	Message Padding	3
2.2	Block Constants	3
2.3	Overall structure	3
2.4	Variant Parameters and Security Strengths	5
3	A Specific PRNG Primitive	5
3.1	Local NLSR Feedback	5
3.1.1	The linear core	5
3.1.2	The nonlinear core	6
3.2	Global Confusion Function	7
3.3	PRNG Interface	9
4	Design Principles	9
4.1	Cryptographic Properties of Components	9
4.1.1	NLSR-Centered Local Feedback	9
4.1.2	Round-Dependent Global Confusion	10
4.1.3	PRNG-Based Hashing Structure	10
4.1.4	S-box Specification	10
4.2	Parameter Selection Rationale	12
5	Provable Security	12
5.1	Ideal PRNG Model	13
5.2	Indifferentiability	13
5.2.1	Simulator Construction	13
5.2.2	Indifferentiability Bound	14
5.3	Classical Security	15
5.3.1	Collision Resistance	16
5.3.2	Preimage Resistance	17
5.3.3	Second-Preimage Resistance	18
5.4	Quantum Security	19
5.4.1	QROM Lifting Lemma Used in the Proof	20
5.4.2	Quantum Collision Resistance	20
5.4.3	Quantum Preimage Resistance	21
5.4.4	Quantum Second-Preimage Resistance	21
6	Other security features	22
6.1	Cryptanalysis of the underlying PRNG	22
6.2	Statistical Randomness Tests	23
6.3	Resistance to Length Extension Attacks	24
7	Performance Evaluation	24
7.1	Benchmark Methodology	25
7.2	Bottleneck Analysis	25
7.3	Future Optimization Directions	26
8	Feature Statement	26
9	Conclusion	27

1 Introduction

Cryptographic hash functions are central to many primitives and protocols, and their security objectives must be re-evaluated in the post-quantum era. Grover-style quantum search reduces generic preimage resistance, encouraging designs that increase internal state and resist structural decomposition.

This proposal explores an alternative to the widely used Merkle–Damgård [4, 7] and sponge [2, 3] paradigms: a construction built from a Cryptographically Secure Pseudorandom Number Generator PRNG. Instead of defining a dedicated compression function, NEULASER treats each message block as a seed/key input to the underlying PRNG and uses disjoint parts of the produced output stream for chaining and final output. The name NEULASER shares the letters N-L-S-R with the NonLinear Shift Register.

2 PRNG-Based Construction

This section specifies the PRNG-based hash construction used by NEULASER. It first defines message padding and block processing, then describes how each chaining value and message block initializes a bounded-output PRNG invocation, how chaining and digest bits are extracted from the generated stream, and finally summarizes the parameter sets together with their classical and quantum security strengths.

2.1 Message Padding

Given a message $M \in \{0,1\}^*$, let $|M|$ denote its bit length. The construction uses padding rule

$$M^* = M \parallel 1 \parallel 0^z \parallel \text{len}_{64}(M),$$

where $\text{len}_{64}(M)$ is the 64-bit binary encoding of $|M|$. The encoding is interpreted in the usual most-significant-bit first order. The integer $z \geq 0$ is chosen as the smallest value satisfying

$$|M| + 1 + z + 64 \equiv 0 \pmod{k}.$$

The padded message is divided into ℓ message blocks

$$M^* = M_0 \parallel M_1 \parallel \cdots \parallel M_{\ell-1},$$

where each block has length k bits, equal to the seed/key input length accepted by the underlying PRNG. In the proposed parameter set, $k = 960, 1216, 1472$ for NEULASER-512, NEULASER-768, and NEULASER-1024 respectively. The terminal length field makes the encoding prefix-free: two distinct messages cannot lead to the same padded bit string.

2.2 Block Constants

To prevent structural symmetries and repeated internal chaining values, each block is associated with a v -bit block constant

$$C_i = (0^{v-64}, i_{\text{high}}, i_{\text{low}}),$$

where

$$i_{\text{high}} = \left\lfloor \frac{i}{2^{32}} \right\rfloor, \quad i_{\text{low}} = i \bmod 2^{32}.$$

Thus $C_i \in \{0,1\}^v$ is the same size as the chaining value. It is xored into the chaining-value part of the construction, while M_i remains the k -bit seed/key input of the PRNG.

2.3 Overall structure

The overall structure of the PRNG-based construction is shown in Figure 1.

The public initialization vector is specified by a single 1088-bit word sequence IV^* . It is represented below as 34 words of 32 bits; the words are encoded in big-endian byte order and concatenated from left to right.

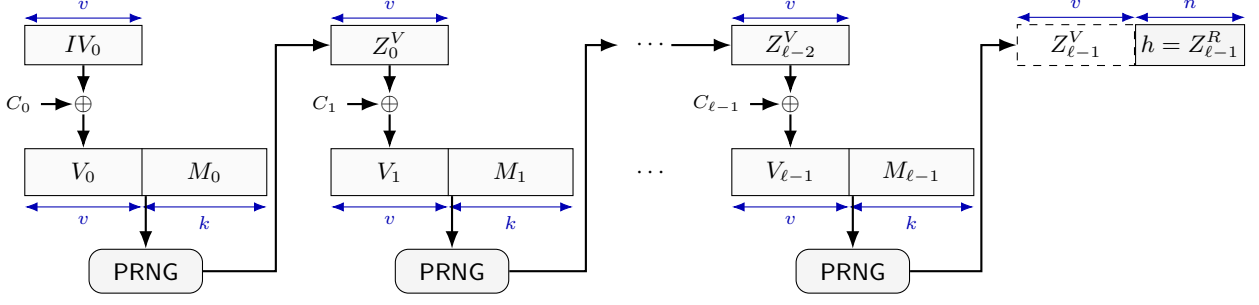


Figure 1: PRNG-based construction.

$IV^* =$

0xF43E9BDE	0x7C2CD551	0x12006E2D	0x8A02761F
0x07F697F7	0xF9D459ED	0xC69C9E7C	0x1928306B
0x942D886D	0xE086856F	0x0A9356EB	0xF1E98E15
0x3490F013	0x2E539BF2	0xA05258CD	0xDE0CBF6F
0xF72EEBA6	0x2DB13B3C	0xF86C80E1	0x529EAE41
0x5CF9DE6F	0xAF848C1	0x7A659972	0x78795D7A
0x18910D89	0x6510F1BD	0x9765E522	0x0F31B6CD
0xFE5AAAE9	0x9E433325	0x43B0477E	0xADA38854
0x0F2F52BE	0x024D6691		

Let $\text{first}_q(IV^*)$ denote the prefix consisting of the first q 32-bit words. The public IVs used by the three versions are

$$IV_{512} = \text{first}_{18}(IV^*), \quad IV_{768} = \text{first}_{26}(IV^*), \quad IV_{1024} = \text{first}_{34}(IV^*) = IV^*.$$

The derivation of these public values is described in Section 4.2.

Let

$$V_0 = IV_n \oplus C_0$$

denote the first chaining value. For each message block M_i , invoke the PRNG as

$$Z_i = \text{PRNG}(V_i, M_i).$$

Although the output stream of PRNG is arbitrarily long, when processing each non-last message block, only the first v bits of Z_i are used in the chaining value for the next block. In the processing of the last message block, output $(v+n)$ bits and the last n bits of $Z_{\ell-1}$ are truncated to form the final hash value. We use Z_i^V to denote the first v bits of the output stream of PRNG and Z_i^R to denote the continued n bits. So we have

$$Z_i = Z_i^V \in \{0,1\}^v, \quad 0 \leq i \leq \ell-2,$$

$$Z_{\ell-1} = Z_{\ell-1}^V || Z_{\ell-1}^R \in \{0,1\}^{v+n}.$$

Then the chaining values are computed as

$$V_{i+1} = Z_i^V \oplus C_{i+1}, \quad 0 \leq i \leq \ell-2.$$

For the last block, the continued n -bit after v output stream bits is the final hash value

$$H_n(M) = \text{HashOut}(Z_{\ell-1}) = Z_{\ell-1}^R.$$

So the function $\text{HashOut}(\cdot)$ is simply the extraction for the last n bits.

Use PRNG_x to denote the first x bits of the output stream of PRNG. Equivalently, the PRNG-based construction can be described as

$$V_0 = IV_n \oplus C_0,$$

$$V_{i+1} = \text{PRNG}_v(V_i, M_i) \oplus C_{i+1}, \quad 0 \leq i \leq \ell-2,$$

and

$$H_n(M) = \text{HashOut}(\text{PRNG}_{v+n}(V_{\ell-1}, M_{\ell-1})).$$

2.4 Variant Parameters and Security Strengths

The NEULASER family contains three fixed-output versions, denoted NEULASER-512, NEULASER-768, and NEULASER-1024. The parameter n is the digest length, k is the message-block length and the seed/key input length of the underlying PRNG, and v is the chaining-value length. The concrete PRNG register size is $k + v$, matched to the size of the coupled NLSR state. The parameters are summarized in Table 1.

The last six columns of Table 1 give the corresponding generic bit-security strength of collision, preimage, and second-preimage resistance in the classical and quantum settings. They are obtained from the proof bounds in Section 5. Since the quantum collision adversary advantage is not tight, the practical security level is believed to be no lower than that achieved when considering general attack. For second-preimage resistance, α denotes the padded block length of the challenge message and we write $\lambda = \log_2 \alpha$.

Version	n	k	v	$k + v$	μ	Col.	Pre.	spre.	q-col	q-pre.	q-spre
NEULASER-512	512	960	576	1536	3	256	512	$\min\{512, 576 - \lambda\}$	128	256	$\min\{256, 288 - \lambda/2\}$
NEULASER-768	768	1216	832	2048	4	384	768	$\min\{768, 832 - \lambda\}$	192	384	$\min\{384, 416 - \lambda/2\}$
NEULASER-1024	1024	1472	1088	2560	5	512	1024	$\min\{1024, 1088 - \lambda\}$	256	512	$\min\{512, 544 - \lambda/2\}$

Table 1: Variant parameters and generic security strengths in bits.

3 A Specific PRNG Primitive

ZUC is a well-studied and successful stream cipher based on a word-oriented register backbone and a compact nonlinear filtering function [1]. Its design shows that modular addition, XOR, rotation-xor diffusion, and byte substitution can be combined efficiently in a stream-generator setting. However, the present hash design requires much larger initialized states, namely 1536, 2048, and 2560 bits, and uses each PRNG invocation only for the bounded output needed by one message block. Thus a direct single-register ZUC-style instance is not well matched to this setting. A natural adaptation is to couple several NLSR modules: the local update reuses ZUC-style word operations, while the global coupling calls the same function components across modules to spread information over the whole large state. Therefore, the proposed PRNG primitive is a coupled NLSR design that reuses the ZUC-style nonlinear word function in both local and global layers.

The underlying PRNG couples $\mu = 3, 4$, and 5 parallel NLSR modules respectively for the three hash function versions. Each module consists of a 16-stage word-oriented NLSR over

$$\mathbb{F}_p, \quad p = 4294967291,$$

where p is a 32-bit prime. Each stage is an element of the finite field $\mathbb{F}_p$. Thus each module has 512 bits of state, so the full internal state has $\mu \cdot 16 \cdot 32$ bits, say 1536, 2048, and 2560 bits. The global state is

$$S = (L^{(0)}, L^{(1)}, \dots, L^{(\mu-1)}),$$

where

$$L^{(i)} = (s_0^{(i)}, s_1^{(i)}, \dots, s_{15}^{(i)}), \quad s_j^{(i)} \in \{0, 1\}^{32}, 0 \leq j < 16, 0 \leq i < \mu.$$

Each NLSR is updated by a local feedback function with inputs from the current module state as well as a global confusion function that mixes information from all μ modules.

3.1 Local NLSR Feedback

The NLSR module is a nonlinear feedback shift register. The local feedback function consists of a linear core and a nonlinear core. For a fixed module i , we write $L^{(i)} = (s_0, s_1, \dots, s_{15})$ in this subsection and suppress the module superscript.

3.1.1 The linear core

The linear core is kept as a low-weight primitive recurrence over $\mathbb{F}_p$, namely

$$\ell_{\text{lin}} = (2^{13} + 1)s_0 + (2^{24} + 1)s_4 + 2^{22}s_9 + (2^3 + 1)s_{13} + 2^{19}s_{15} \pmod{p}.$$

Equivalently,

$$\ell_{\text{lin}} = 8193s_0 + 16777217s_4 + 4194304s_9 + 9s_{13} + 524288s_{15} \pmod{p}.$$

The corresponding characteristic polynomial is

$$\chi_{\text{lin}}(x) = x^{16} - 524288x^{15} - 9x^{13} - 4194304x^9 - 16777217x^4 - 8193.$$

The polynomial $\chi_{\text{lin}}(x)$ is primitive over $\mathbb{F}_p$. Therefore, for any nonzero initial state, the linear core alone generates a maximum-length sequence, i.e., an m -sequence, with period

$$p^{16} - 1.$$

3.1.2 The nonlinear core

To destroy the purely linear recurrence, a nonlinear word function is computed from four state words. We select the two endpoint words and two middle words:

$$R_0 = s_{15}, \quad R_1 = s_{10}, \quad R_2 = s_5, \quad R_3 = s_0.$$

The data flow of the local feedback computation is shown in Figure 2. The dashed rectangle represents the nonlinear word function F , while the upper part shows the linear core and the final reduction modulo p .

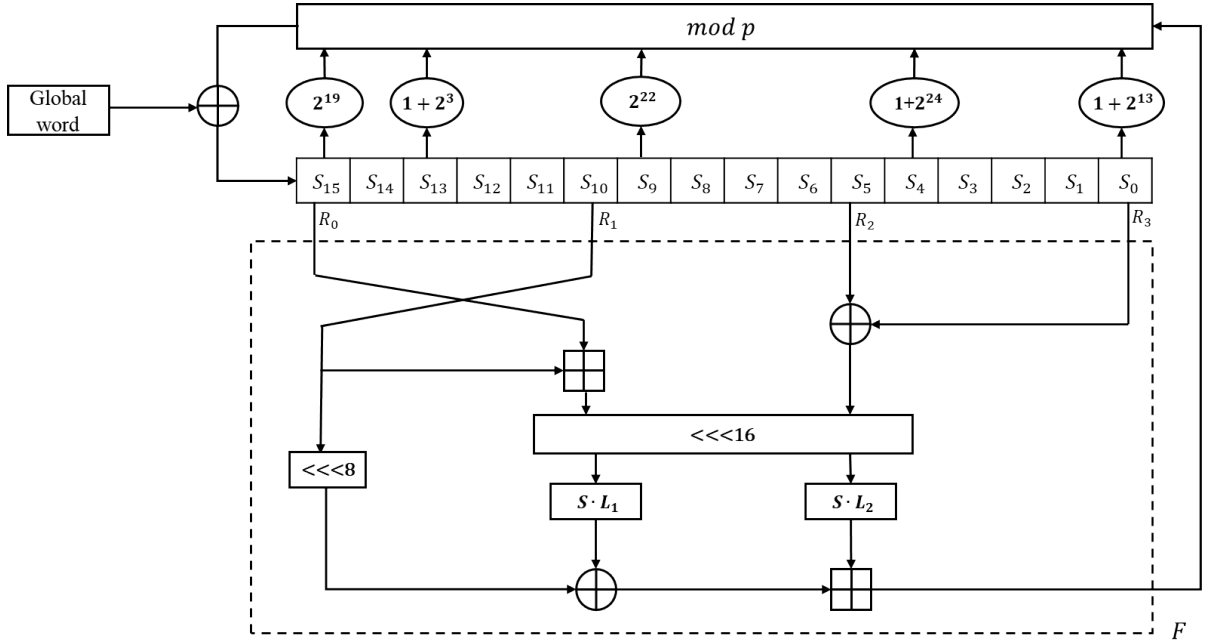


Figure 2: Local NLSR feedback computation.

All bitwise operations are applied to the canonical 32-bit representations of field elements. First form two 32-bit words

$$A = (R_0 + R_1) \bmod 2^{32}, \quad B = R_2 \oplus R_3.$$

Concatenate them into a 64-bit word and rotate it left by 16 bits:

$$U \parallel V = \text{rotl}_{64}(A \parallel B, 16), \quad U, V \in \{0, 1\}^{32}.$$

The left half U and the right half V are processed by two word-oriented linear transformations reused from ZUC [1], followed by the same parallel S-box layer. For a 32-bit word X , define

$$\mathcal{L}_1(X) = X \oplus \text{rotl}_{32}(X, 2) \oplus \text{rotl}_{32}(X, 10) \oplus \text{rotl}_{32}(X, 18) \oplus \text{rotl}_{32}(X, 24),$$

and

$$\mathcal{L}_2(X) = X \oplus \text{rotl}_{32}(X, 8) \oplus \text{rotl}_{32}(X, 14) \oplus \text{rotl}_{32}(X, 22) \oplus \text{rotl}_{32}(X, 30).$$

These two rotation-xor layers provide lightweight word-level diffusion and avoid the implementation cost of byte-field matrix multiplication.

The parallel S-box layer is

$$\mathcal{S}(X_0 \| X_1 \| X_2 \| X_3) = S_E(X_0) \| S_E(X_1) \| S_E(X_2) \| S_E(X_3),$$

where S_E is the fixed 8-bit S-box specified in Section 4.1.4. Let

$$P_L = \mathcal{S}(\mathcal{L}_1(U)), \quad P_R = \mathcal{S}(\mathcal{L}_2(V)).$$

The local nonlinear filter is then

$$F(R_0, R_1, R_2, R_3) = ((P_L \oplus \text{rotl}_{32}(R_1, 8)) + P_R) \bmod 2^{32}.$$

The local NLSR feedback word is defined by adding this nonlinear word to the linear core modulo p :

$$\Phi(L^{(i)}) = \text{red}_p(\ell_{\text{lin}} + F(s_{15}, s_{10}, s_5, s_0)),$$

where

$$\text{red}_p(x) = x \bmod p.$$

3.2 Global Confusion Function

The μ NLSR modules are mixed in every state update by a global confusion function built from the same nonlinear word function F used in the local feedback. This is designed to reuse the implementation of F to be efficient.

Let the state of module i be

$$L^{(i)} = (s_0^{(i)}, s_1^{(i)}, \dots, s_{15}^{(i)}), \quad 0 \leq i < \mu.$$

For each module define four extracted words

$$A_i = s_{14}^{(i)}, \quad B_i = s_9^{(i)}, \quad C_i = s_6^{(i)}, \quad D_i = s_1^{(i)}.$$

These four words will form different input words of F in different rounds. The module indices are taken modulo μ . Let

$$[x]_\mu = x \bmod \mu$$

and let $r \geq 0$ denote the round number. Each round uses a μ -dependent offset table, and the round dependence is introduced by adding r to the selected offsets. Thus an offset d from module i in round r refers to module $[i + r + d]_\mu$.

The base offset table for the inputs of F is

μ	(a_X, b_X)	(a_Y, b_Y)	(a_Z, b_Z)	(a_W, b_W)
3	(0, 1)	(1, 2)	(2, 0)	(1, 0)
4	(0, 1)	(2, 0)	(1, 3)	(3, 2)
5	(0, 1)	(2, 4)	(3, 1)	(4, 3)

The pair (a, b) means that the two words are read from modules $[i + r + a]_\mu$ and $[i + r + b]_\mu$. The table avoids self-pairs. For $\mu = 4$, every offset appears exactly twice. For $\mu = 3$ and $\mu = 5$, the appearances are as balanced as possible with eight offset slots. The directed differences $b - a \bmod \mu$ cover both nonzero differences for $\mu = 3$, cover 1, 2, 3 for $\mu = 4$, and cover all nonzero differences for $\mu = 5$.

The round- r cross-module inputs to F are

$$X_r^{(i)} = A_{[i+r+a_X]_\mu} \oplus \text{rotl}(B_{[i+r+b_X]_\mu}, 7),$$

$$Y_r^{(i)} = (B_{[i+r+a_Y]_\mu} + C_{[i+r+b_Y]_\mu}) \bmod 2^{32},$$

$$Z_r^{(i)} = C_{[i+r+a_Z]_\mu} \oplus \text{rotl}(D_{[i+r+b_Z]_\mu}, 11),$$

and

$$W_r^{(i)} = (D_{[i+r+a_W]_\mu} + A_{[i+r+b_W]_\mu}) \bmod 2^{32}.$$

The global nonlinear word is

$$\Gamma_r^{(i)} = F(X_r^{(i)}, Y_r^{(i)}, Z_r^{(i)}, W_r^{(i)}) \oplus \kappa_r^{(i)},$$

where $\kappa_r^{(i)}$ is a fixed public 32-bit constant for round r and module i . Concretely,

$$\kappa_r^{(i)} = \text{rotl}(0\mathbf{x}9\mathbf{e}3779\mathbf{b}9 \oplus r \oplus i, (r + i) \bmod 32),$$

with r and i encoded as 32-bit words. The XOR with the public counters r and i , followed by the rotation by $(r + i) \bmod 32$, only provides round and module separation.

The vector

$$\Gamma_r(S) = (\Gamma_r^{(0)}, \Gamma_r^{(1)}, \dots, \Gamma_r^{(\mu-1)})$$

is the round- r global confusion output.

The feedback word inserted into module i is

$$f_r^{(i)} = \text{red}_p(\Phi(L^{(i)}) \oplus \Gamma_r^{(i)}).$$

Besides, $\Gamma_r^{(i)}$ s are also injected into three internal tap positions. Then each NLSR is shifted and updated as

$$L'_i = (t_0^{(i)}, t_1^{(i)}, \dots, t_{15}^{(i)}),$$

where

$$t_j^{(i)} = s_{j+1}^{(i)} \quad \text{for } j \notin \{3, 7, 11, 15\},$$

and the global confusion words are also injected into three internal tap positions. Let

$$(e_1, e_2, e_3) = \begin{cases} (1, 2, 1), & \mu = 3, \\ (1, 2, 3), & \mu = 4 \text{ or } \mu = 5. \end{cases}$$

Then

$$t_3^{(i)} = \text{red}_p(s_4^{(i)} \oplus \text{rotl}(\Gamma_r^{([i+e_1]_\mu)}, 5)),$$

$$t_7^{(i)} = \text{red}_p(s_8^{(i)} \oplus \text{rotl}(\Gamma_r^{([i+e_2]_\mu)}, 13)),$$

$$t_{11}^{(i)} = \text{red}_p(s_{12}^{(i)} \oplus \text{rotl}(\Gamma_r^{([i+e_3]_\mu)}, 21)),$$

$$t_{15}^{(i)} = f_r^{(i)}.$$

The full state update is

$$S' = \mathcal{U}_r(S) = (L'_0, L'_1, \dots, L'_{\mu-1}).$$

Every round simultaneously advances each NLSR, injects the local nonlinear feedback, and mixes information from all μ modules through the F -based global confusion layer. A PRNG evaluation applies $S_{r+1} = \mathcal{U}_r(S_r)$, so the cross-module wiring changes with r rather than remaining fixed throughout the execution.

3.3 PRNG Interface

The coupled NLSR primitive is used as a Cryptographically Secure Pseudorandom Number Generator

$$\text{PRNG} : \mathcal{V} \times \mathcal{K} \rightarrow \{0, 1\}^\infty.$$

On input an initialization vector V and a seed/key K , the implementation initializes the (512μ) -bit coupled NLSR state from (V, K) . Let the initialized state be S_0 , and let

$$N_{\text{blank}} = 32.$$

First run N_{blank} blank rounds:

$$S_{r+1} = \mathcal{U}_r(S_r), \quad 0 \leq r < N_{\text{blank}}.$$

After the blank rounds, the output stream is generated in 32-bit words. For output step $\tau = 0, 1, 2, \dots$, first update the state once more:

$$S_{N_{\text{blank}}+\tau+1} = \mathcal{U}_{N_{\text{blank}}+\tau}(S_{N_{\text{blank}}+\tau}).$$

Write the updated module i at this step as

$$L_\tau^{(i)} = (s_{0,\tau}^{(i)}, s_{1,\tau}^{(i)}, \dots, s_{15,\tau}^{(i)}).$$

The output word of module i at step τ is obtained by a lightweight compression of fixed output taps using XOR and modular addition:

$$O_\tau^{(i)} = ((s_{13,\tau}^{(i)} \oplus s_{8,\tau}^{([i+1]\mu)}) + (s_{4,\tau}^{([i+2]\mu)} \oplus s_{2,\tau}^{([i+3]\mu)})) \bmod 2^{32}.$$

The cross-module offsets in the formula make the output word depend on several modules before it is exposed. The τ -th output block is

$$O_\tau = O_\tau^{(0)} \parallel O_\tau^{(1)} \parallel \dots \parallel O_\tau^{(\mu-1)} \in \{0, 1\}^{32\mu}.$$

The infinite output stream is

$$\text{PRNG}(V, K) = O_0 \parallel O_1 \parallel O_2 \parallel \dots$$

For a requested output length λ , define

$$\text{PRNG}_\lambda(V, K) = \text{first}_\lambda(\text{PRNG}(V, K)),$$

where first_λ denotes the leftmost λ bits. In the hash mode, each invocation is specified with at most $\lambda = v + n$. To processing non-final message blocks only $\lambda = v$ bits are generated for chaining. For the last block, $\lambda = v + n$ bits are generated and the rightmost n bits are truncated as the hash value. The hash mode treats this generator as a black-box cryptographically secure pseudorandom primitive.

4 Design Principles

4.1 Cryptographic Properties of Components

4.1.1 NLSR-Centered Local Feedback

The local feedback is designed to combine an analyzable shift-register backbone with compact nonlinear disruption. The linearized recurrence is chosen to have a primitive characteristic polynomial over $\mathbb{F}_p$, so the underlying register motion has maximum period before nonlinear terms are added. This property is not used as a stand-alone security claim, but it prevents short linear cycles from being built into the state update and gives a stable long-period skeleton for diffusion.

The nonlinear part is introduced to destroy the algebraic simplicity of that skeleton. It mixes words of different ages through heterogeneous operations: field addition, addition modulo 2^{32} , xors, rotations, byte-wise substitution, and rotation-xor diffusion. Reusing the ZUC linear transformations gives simple and well-studied word-level diffusion, so an active difference is mixed across bit and byte positions before the nonlinear word is reinserted into the register. When the input and output words are viewed as four-byte vectors, both $\mathcal{L}_1$ and $\mathcal{L}_2$ have differential and linear branch number 5.

The design goal of the local layer is therefore threefold: retain the efficiency and long-state behavior of word-oriented shift registers, prevent the update from being modeled as a low-degree or purely linear recurrence, and provide fast intra-module diffusion using operations that are natural on 32-bit platforms.

4.1.2 Round-Dependent Global Confusion

The coupled NLSR modules should not decompose into several nearly independent streams. The purpose of the global confusion layer is to turn local evolution into genuinely joint evolution: information from one module is continually mixed into the update path of the others, so that correlations or partial guesses cannot be confined to a single register for many rounds.

The round-dependent module schedule is included to avoid a fixed wiring pattern, especially for the small values $\mu = 3, 4, 5$. A static offset rule can accidentally create repeated pairings or short interaction cycles after reduction modulo μ . Adding the round number to the public offset table keeps the rule simple while changing the cross-module neighborhood from round to round. This improves temporal diversity and helps rule out obvious symmetry classes among modules.

The global layer also uses taps different from those used by the local nonlinear update and by the output extraction. This gives the primitive several views of the register trajectory: local feedback, cross-module coupling, and output compression do not all observe the same word positions. Public round/module constants provide domain separation and break residual symmetries; they are not secret, and their role is to make the deterministic coupling non-translation-invariant across rounds and modules.

Thus the intended properties of the global layer are cross-module diffusion, resistance to decomposition and guess-and-determine strategies, avoidance of fixed small- μ wiring cycles, and preservation of a compact implementation by reusing the same nonlinear word function as the local layer.

4.1.3 PRNG-Based Hashing Structure

The structural idea of NEULASER is to build the hash function on top of a cryptographically secure PRNG rather than on a dedicated compression function or permutation. The security intuition is that, once the pair consisting of the current chaining value and the current message block initializes a secure PRNG invocation, the produced stream should be indistinguishable from fresh random bits except through the intended chaining path. The hash construction therefore uses the pseudorandomness of the underlying generator as the main source of compression security, chaining security, and final-output unpredictability.

This gives a different point in the design space of hash functions. The construction is close to the Merkle–Damgård philosophy [4, 7] in that it processes a padded message block by block and maintains an explicit chaining value. In this sense it can be viewed as a generalized MD-type construction. However, the transition from one chaining value to the next is not a conventional fixed-output compression function; it is the prefix of a PRNG output stream generated from a large initialized state. The final digest is then taken from a separate continued part of the same final stream.

The construction is also related to sponge-style hashing [2, 3]. Like a sponge, it separates absorbed information from squeezed output and relies on a large internal state to hide intermediate structure. Unlike the standard sponge, the primitive interface is a PRNG oracle with a seed/key input and a public chaining input, rather than a public permutation repeatedly applied to one state. For this reason the formal proof is written for an ideal PRNG oracle, but many of the proof ideas are closer to sponge indistinguishability arguments: one tracks which paths are already determined by oracle queries and shows that only saturated, consistently reconstructed paths can reveal a difference between the real and ideal worlds.

The next chapter gives the provable-security analysis. The purpose of this subsection is only to explain the design rationale: the construction is intended to transfer the cryptographic strength of the PRNG to the hash mode, while retaining a simple iterative structure and a proof strategy compatible with modern sponge-based reasoning.

4.1.4 S-box Specification

The byte substitution layer uses a single fixed 8-bit S-box constructed from inversion over a binary extension field followed by an affine transformation. Let

$$g_5(X) = X^8 + X^5 + X^3 + X^2 + 1$$

be an irreducible polynomial over $\mathbb{F}_2$, and identify each byte

$$x = (x_7, \dots, x_0)$$

with the field element

$$x_7X^7 + \dots + x_1X + x_0 \in \mathbb{F}_{2^8} = \mathbb{F}_2[X]/(g_S(X)).$$

Define the inversion layer

$$I(x) = \begin{cases} 0, & x = 0, \\ x^{-1}, & x \neq 0, \end{cases}$$

where inversion is taken in $\mathbb{F}_{2^8}$. For

$$u = I(x) = (u_7, \dots, u_0),$$

define the affine output

$$y_i = u_i \oplus u_{i+1} \oplus u_{i+3} \oplus u_{i+4} \oplus u_{i+6} \oplus c_i, \quad 0 \leq i < 8,$$

where indices are taken modulo 8 and

$$(c_7, \dots, c_0) = \text{0xa7}.$$

The S-box is

$$S_E(x) = (y_7, \dots, y_0).$$

This choice uses a field polynomial and affine layer different from AES, and its lookup table is also different from the SM4 S-box.

The complete lookup table for S_E is given in Table 2.

Table 2: NEULASER S-box Lookup Table

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x	A7	12	9B	70	B9	EA	4C	73	4E	14	01	33	52	31	2B	9C
1x	B5	F5	98	21	F4	BE	0B	2A	BB	AC	EC	8D	E1	90	3A	38
2x	48	D2	68	0F	DE	56	E4	0E	E8	ED	CD	E5	F1	05	61	86
3x	A9	13	22	2D	02	9E	B2	7E	84	DC	3C	1E	69	E7	8E	EB
4x	B6	D8	FB	BF	A6	4A	15	23	FD	A4	5F	71	E0	8B	95	B8
5x	E6	08	82	34	92	65	60	E9	8C	AB	10	C1	C4	0C	37	4F
6x	46	D6	1B	30	83	7B	E2	A5	93	3F	DD	CF	CB	F7	AD	B3
7x	D0	47	1A	C6	6A	B0	9D	7D	26	64	87	25	D5	99	81	FA
8x	2F	28	FE	EE	89	39	4D	F6	27	F2	51	19	18	D9	03	7F
9x	8A	D1	C0	43	3D	C3	CC	C5	04	55	B1	3E	58	BC	CE	AA
Ax	07	42	96	77	D3	6E	88	45	DB	1D	20	6F	44	DF	66	A8
Bx	32	54	A1	0A	7C	63	94	97	F0	76	72	6B	EF	3B	35	80
Cx	57	74	1F	36	F9	AF	6C	06	53	24	C9	F3	E3	C7	40	FC
Dx	5B	16	0D	DA	9A	C8	75	F8	91	BD	8F	59	A2	AE	4B	A3
Ex	1C	FF	D7	50	79	2E	17	85	41	9F	2C	5C	BA	7A	CA	6D
Fx	67	11	A0	C2	B7	29	00	49	78	62	5E	5A	B4	D4	09	5D

Since inversion over $\mathbb{F}_{2^8}$ is followed only by an invertible affine transformation, S_E is bijective. Its differential uniformity is

$$\delta(S_E) = 4,$$

which is optimal for 8-bit bijective S-boxes. Its maximum Walsh amplitude is

$$\max_{a,b \neq 0} |W_{S_E}(a,b)| = 32,$$

equivalently its component-function nonlinearity is

$$2^7 - \frac{32}{2} = 112,$$

which is also optimal for 8-bit bijective S-boxes.

4.2 Parameter Selection Rationale

The field prime

$$p = 4294967291$$

is the largest 32-bit prime below 2^{32} . It permits efficient word-oriented arithmetic while avoiding a purely binary linear recurrence over $\mathbb{F}_{2^{32}}$. The 16-stage length gives a 512-bit module; matching $\mu = 3, 4, 5$ modules with the three variants gives

$$v + k = 512\mu \in \{1536, 2048, 2560\}.$$

Thus the PRNG register size is exactly the coupled-NLSR state size.

The digest lengths are $n = 512, 768, 1024$. The corresponding block lengths $k = 960, 1216, 1472$ and chaining lengths $v = 576, 832, 1088$ are selected so that $k + v$ equals the NLSR state size in each variant. This keeps the initialization interface simple: the whole coupled state is filled by the message-block input and chaining-value input, without introducing a hidden state-size conversion layer.

The local linear coefficients are powers of two, or powers of two plus one, so multiplication by them can be implemented efficiently with shifts and additions before reduction modulo p . At the same time, the associated 16-stage characteristic polynomial is primitive over $\mathbb{F}_p$, giving the linear core a maximum-period backbone. The nonlinear F layer, the ZUC-style rotation-xor linear layers, and the S-box layer are then added to increase algebraic complexity and diffusion.

Public constants are chosen transparently. In particular, the global confusion constant uses the golden-ratio word

$$\gamma = \left\lfloor 2^{32} \frac{\sqrt{5} - 1}{2} \right\rfloor = \text{0x9e3779b9},$$

and the remaining round and module variation is obtained from the public indices r and i .

The public initialization vector IV^* in Section 2 is generated by the same nothing-up-my-sleeves rule. Let

$$\eta = \left\lfloor 2^{32}(\sqrt{2} - 1) \right\rfloor = \text{0x6a09e667}.$$

Define

$$\eta^* = \eta \oplus 1024,$$

where 1024 is encoded as a 32-bit public word. For $0 \leq j < 34$, the j -th 32-bit word of IV^* is

$$IV_j^* = \text{rotr}_{32}((\gamma \cdot (j + 1)) \bmod 2^{32}, j \bmod 32) \oplus \text{rotr}_{32}(\eta^*, (7j) \bmod 32).$$

Finally,

$$IV^* = IV_0^* \parallel IV_1^* \parallel \cdots \parallel IV_{33}^*.$$

The three variants then take prefixes of this sequence:

$$(q_{512}, q_{768}, q_{1024}) = (18, 26, 34), \quad IV_n = \text{first}_{q_n}(IV^*).$$

This rule uses only fixed mathematical constants and public parameters. The same primitive, constants, and hash mode are used by the three versions; only the public parameter set and the IV prefix length change.

5 Provable Security

This section analyzes the hash mode from Section 2. We first define the ideal PRNG model used in the proofs. We then prove indistinguishability from a random oracle, and derive classical and quantum bounds for collision resistance, preimage resistance, and second-preimage resistance. For the concrete construction, each bound should add the distinguishing advantage against the actual coupled-NLSR PRNG instantiation.

5.1 Ideal PRNG Model

For convenience in the proof, we omit the effect of the block constant C_i . This gives a weaker, more conservative proof model than the concrete construction.

In the concrete hash construction, the PRNG may be viewed as producing an arbitrarily long output stream. However, only two disjoint parts of this stream are relevant: during message processing the construction uses v bits to form the next chaining value, and during final hash extraction it uses another n bits to form the digest material. All other output bits are ignored by the hash construction and are not made available to the adversary in this proof. Therefore the total output length exposed per PRNG invocation is $v + n$ bits. The input material consists of a k -bit message block together with a v -bit chaining value. Thus the ideal proof models the accessible PRNG transcript as a fixed-length random function

$$\mathcal{R}_{\text{PRNG}} : \{0, 1\}^{v+k} \rightarrow \{0, 1\}^{v+n}.$$

For each distinct proof-level input string, the oracle returns an independent uniformly random $(v+n)$ -bit string and maintains consistency for repeated queries. The generated output stream is partitioned into two disjoint components

$$Z_i = Z_i^V \parallel Z_i^R, Z_i^V \in \{0, 1\}^v, Z_i^R \in \{0, 1\}^n.$$

The first component is used as the chaining value for the next block:

$$V_{i+1} = \text{Link}(Z_i) = Z_i^V.$$

So the function $\text{Link}(\cdot)$ is simply the extraction for the first v bits.

5.2 Indifferentiability

We first recall the indifferentiability notion used in this proof. Let $C[F]$ be a construction using an ideal primitive F , and let G be an ideal primitive with the same external interface as $C[F]$. The construction $C[F]$ is indifferentiable from G if there exists a simulator $P[G]$, which has oracle access to G and exposes the same interface as F , such that no distinguisher can tell apart the two systems

$$(C[F], F) \quad \text{and} \quad (G, P[G]).$$

For a distinguisher D , the indifferentiability advantage is

$$\text{Adv}_{C[F], G}^{\text{indiff}}(D) = \left| \Pr[D^{C[F], F} = 1] - \Pr[D^{G, P[G]} = 1] \right|.$$

In this document, $C[F]$ is the hash construction $H[\mathcal{R}_{\text{PRNG}}]$, the ideal primitive F is the ideal PRNG oracle $\mathcal{R}_{\text{PRNG}}$, and G is a random oracle $\mathcal{RO}$. Therefore the goal is to construct $P[\mathcal{RO}]$ and bound

$$\text{Adv}_{H[\mathcal{R}_{\text{PRNG}}], \mathcal{RO}}^{\text{indiff}}(D) = \left| \Pr[D^{H[\mathcal{R}_{\text{PRNG}}], \mathcal{R}_{\text{PRNG}}} = 1] - \Pr[D^{\mathcal{RO}, P[\mathcal{RO}]} = 1] \right|.$$

The proof below shows that this advantage is bounded by the statistical distance introduced by the simulator's saturated set of chaining values. The simulator and saturated-set argument follow the indifferentiability proof strategy for the sponge construction [3], adapted here to the proof-level PRNG oracle and rooted chaining paths.

5.2.1 Simulator Construction

Let $\mathcal{RO}$ denote a random oracle. The simulator $P[\mathcal{RO}]$ maintains a table

$$T_{\text{PRNG}}[Q]$$

for simulated PRNG outputs indexed by the proof-level PRNG input Q , and a table

$$T_{\text{path}}[V]$$

for rooted chaining paths. A rooted chaining path is a sequence of valid proof-level PRNG invocations that starts from the public root V_0 and follows the chaining rule $V_{i+1} = \text{Link}(Z_i)$. If $T_{\text{path}}[V] = X$, then V is a chaining value reachable from the root by processing the message prefix X . The simulator also maintains two sets

$$\text{Root} = \{V : T_{\text{path}}[V] \text{ is defined}\}$$

and

$$\text{Out} = \{V : \text{some right-interface query } (K, V) \text{ has already been answered}\}.$$

The set Root contains chaining values with known rooted paths, while Out contains chaining values from which an outgoing simulated PRNG edge already exists. Following the saturated-set argument used for the sponge construction [3], a rooted query may safely create a new rooted node only outside

$$\text{Sat} = \text{Root} \cup \text{Out}.$$

The simulator is called saturated if $\text{Sat} = \{0, 1\}^v$. Initially,

$$T_{\text{path}}[V_0] = \epsilon.$$

Hence $\text{Root} = \{V_0\}$ and $\text{Out} = \emptyset$. Whenever the adversary queries a PRNG input Q , the simulator parses it as the corresponding pair (K, V) . If the query already exists in T_{PRNG} , the stored answer is returned. Otherwise, if V corresponds to a rooted path, the simulator reconstructs the corresponding message prefix

$$X = T_{\text{path}}[V] \parallel K.$$

It then samples a fresh chaining value

$$V' \leftarrow \{0, 1\}^v \setminus \text{Sat}$$

as long as the simulator is not saturated. This exclusion has two purposes. First, $V' \notin \text{Root}$ prevents two different rooted paths from reaching the same chaining value. Second, $V' \notin \text{Out}$ prevents a previously answered non-root query from later becoming part of a rooted path with an output component that was not programmed from the random oracle. The output component is defined as

$$R = \mathcal{RO}(X).$$

The simulator returns

$$Z = V' \parallel R,$$

stores the result in T_{PRNG} , and records

$$T_{\text{path}}[V'] = X.$$

It also updates $\text{Root} \leftarrow \text{Root} \cup \{V'\}$ and $\text{Out} \leftarrow \text{Out} \cup \{V\}$. For all other queries, the simulator simply returns a uniformly random $(v + n)$ -bit string and updates $\text{Out} \leftarrow \text{Out} \cup \{V\}$.

5.2.2 Indifferentiability Bound

Theorem 1 (Indifferentiability). *Let $H[\mathcal{R}_{\text{PRNG}}]$ denote the hash construction instantiated with an ideal fixed-length random function $\mathcal{R}_{\text{PRNG}}$. For any distinguisher D making total cost at most $N < 2^v$, there exists a simulator $P[\mathcal{RO}]$ such that*

$$\text{Adv}_{H[\mathcal{R}_{\text{PRNG}}], \mathcal{RO}}^{\text{indiff}}(D) \leq \frac{N(N+1)}{2^{v+1}}.$$

Proof. We prove the theorem using the simulator described above and the saturated-set argument for rooted chaining paths.

We compare the real system

$$\text{Real} = (H[\mathcal{R}_{\text{PRNG}}], \mathcal{R}_{\text{PRNG}})$$

with the ideal system

$$\text{Ideal} = (\mathcal{RO}, P[\mathcal{RO}]).$$

The left interface is either $H[\mathcal{R}_{\text{PRNG}}]$ in the real system or $\mathcal{RO}$ in the ideal system. The right interface is either the ideal PRNG oracle $\mathcal{R}_{\text{PRNG}}$ in the real system or the simulator $P[\mathcal{RO}]$ in the ideal system.

Every rooted chaining path beginning at V_0 corresponds to a unique message prefix as long as the simulator is not saturated. In the ideal system, when $P[\mathcal{RO}]$ receives a rooted query $Q = (K, V)$ with $T_{\text{path}}[V] = X_0$, it sets

$$X = X_0 \parallel K$$

and returns

$$Z = V' \parallel \mathcal{RO}(X).$$

Consequently, if the distinguisher reconstructs the message X through right-interface queries to $P[\mathcal{RO}]$, the n -bit output component obtained on that rooted path is exactly the same value as the left-interface answer $\mathcal{RO}(X)$. This is the precise sense in which the simulator is consistent with the random oracle on rooted paths.

In the real system, the same rooted query is answered by

$$\mathcal{R}_{\text{PRNG}}(K, V) = V' \parallel R,$$

where V' and R are uniform and independent for every fresh proof-level input. The only structural difference is that $P[\mathcal{RO}]$ samples V' outside Sat , whereas $\mathcal{R}_{\text{PRNG}}$ samples V' over all 2^v chaining values. Thus the distinguisher can separate the two systems only when a real rooted transition lands in the saturated set Sat . This event includes an ordinary collision with an already rooted chaining value, and landing on a chaining value from which the distinguisher has already obtained an outgoing right-interface answer.

Define the event SatHit as the occurrence, in the real system, of a rooted right-interface query whose next chaining value belongs to the current set $\text{Sat} = \text{Root} \cup \text{Out}$. Suppose the adversary causes at most N PRNG invocations in total. Before the j -th fresh right-interface answer is returned, the set Sat has size at most j , since every query can enlarge $\text{Root} \cup \text{Out}$ by at most one new chaining value and initially it contains only V_0 . Hence

$$\Pr[\text{SatHit}] \leq 1 - \prod_{j=1}^N \left(1 - \frac{j}{2^v}\right) < \frac{N(N+1)}{2^{v+1}}.$$

Let $\text{View}_{\text{real}}$ and $\text{View}_{\text{ideal}}$ denote the complete transcript distributions seen by the distinguisher in the real and ideal systems. Let

$$\Delta(\text{View}_{\text{real}}, \text{View}_{\text{ideal}}) = \frac{1}{2} \sum_{\tau} |\Pr[\text{View}_{\text{real}} = \tau] - \Pr[\text{View}_{\text{ideal}} = \tau]|$$

be their total variation distance. Couple the real and ideal systems as follows. For non-root queries, use the same uniform $(v+n)$ -bit answer in both systems. For rooted queries, use the same n -bit value for R and $\mathcal{RO}(X)$, and use the same v -bit value for V' whenever the real value lies outside Sat . If the real value lies in Sat , the coupling fails and the event SatHit occurs. Therefore, by the coupling characterization of total variation distance,

$$\Delta(\text{View}_{\text{real}}, \text{View}_{\text{ideal}}) \leq \Pr[\text{SatHit}] < \frac{N(N+1)}{2^{v+1}}.$$

For any distinguisher D ,

$$|\Pr[D(\text{View}_{\text{real}}) = 1] - \Pr[D(\text{View}_{\text{ideal}}) = 1]| \leq \Delta(\text{View}_{\text{real}}, \text{View}_{\text{ideal}}),$$

which gives the claimed advantage bound. The condition $N < 2^v$ ensures that the simulator has at least one chaining value outside Sat whenever it must answer a rooted query before saturation. $\square$

5.3 Classical Security

Following the fixed-output version of the definitions used for permutation-based hashing, the adversary is given oracle access to the underlying component. Let

$$\mathfrak{F}_{v,k,n} = \{F : \{0,1\}^{v+k} \rightarrow \{0,1\}^{v+n}\}$$

be the set of all functions with the same proof-level domain and range as the ideal PRNG. In the following experiments, the underlying component is sampled as

$$F \leftarrow \mathfrak{F}_{v,k,n}$$

uniformly at random, and $H_n[F]$ denotes the hash construction instantiated with this sampled component. We consider collision, preimage, and second preimage resistance of H_n based on random $F \leftarrow \mathfrak{F}_{v,k,n}$, where we take collision resistance, (everywhere) preimage resistance, and (everywhere) second preimage of Rogaway and Shrimpton [8].

In the collision-resistance experiment, A has oracle access to F and outputs two messages $M \neq M'$. Its advantage is

$$\text{Adv}_{H_n}^{\text{coll}}(A) = \Pr_{F \leftarrow \mathfrak{F}_{v,k,n}} [A^F \Rightarrow (M, M') : M \neq M' \wedge H_n[F](M) = H_n[F](M')].$$

The everywhere preimage-resistance advantage is the maximum, over all n -bit challenge images Y , of the probability that A finds a preimage:

$$\text{Adv}_{H_n}^{\text{pre}}(A) = \max_{Y \in \{0,1\}^n} \Pr_{F \leftarrow \mathfrak{F}_{v,k,n}} [A^F(Y) \Rightarrow M : H_n[F](M) = Y].$$

For second-preimage resistance, first define the pointwise advantage for a fixed challenge message M :

$$\text{Adv}_{H_n}^{M-2\text{pre}}(A) = \Pr_{F \leftarrow \mathfrak{F}_{v,k,n}} [A^F(M) \Rightarrow M' : M' \neq M \wedge H_n[F](M') = H_n[F](M)].$$

Let κ denote the maximum allowed length of the first preimage. The everywhere second-preimage-resistance advantage is

$$\text{Adv}_{H_n}^{2\text{pre}[\kappa]}(A) = \max_{M \in \{0,1\}^{\leq \kappa}} \text{Adv}_{H_n}^{M-2\text{pre}}(A).$$

When a theorem below is stated for a fixed challenge message M , it gives the corresponding pointwise bound used in this maximum.

5.3.1 Collision Resistance

We now derive collision resistance of the proposed PRNG-based hash construction from its indistinguishability from a random oracle. Let $C[F]$ denote the hash construction instantiated with an ideal PRNG oracle

$$F : \{0,1\}^{v+k} \rightarrow \{0,1\}^{v+n},$$

and let $H_n[F]$ denote the corresponding n -bit hash function. Assume that $H_n[F]$ is $(t_D, t_S, q, \varepsilon_{\text{ind}})$ -indifferentiable from a random oracle

$$RO : \{0,1\}^* \rightarrow \{0,1\}^n.$$

That is, there exists a simulator $\mathcal{S}^{RO}$ such that for every distinguisher D making at most q total queries,

$$\left| \Pr_F [D^{H_n[F], F} = 1] - \Pr_{RO} [D^{RO, \mathcal{S}^{RO}} = 1] \right| \leq \varepsilon_{\text{ind}}.$$

Theorem 2 (Classical Collision Resistance). *Let A be a classical adversary that makes at most q_H queries to the hash interface $H_n[F]$ and at most q_F queries to the ideal PRNG oracle F . Let $q = q_H + q_F + 2$, where the additional two hash queries are used by the reduction to verify the final collision output. Then*

$$\text{Adv}_{H_n}^{\text{coll}}(A) \leq \varepsilon_{\text{ind}} + \frac{(q_H + 2)(q_H + 1)}{2^{n+1}}.$$

In particular, up to constant factors,

$$\text{Adv}_{H_n}^{\text{coll}}(A) \leq \varepsilon_{\text{ind}} + O\left(\frac{q_H^2}{2^n}\right).$$

Proof. Suppose there exists a collision-finding adversary A against $H_n[F]$. We construct an indistinguishability distinguisher D^A as follows. The distinguisher D^A is given oracle access to a pair of interfaces $(\mathcal{O}_1, \mathcal{O}_2)$. It runs A , answering all hash queries of A by forwarding them to $\mathcal{O}_1$, and all primitive queries of A by forwarding them to $\mathcal{O}_2$. Eventually, A outputs two messages (M, M') . The distinguisher then queries $\mathcal{O}_1(M)$ and $\mathcal{O}_1(M')$, if these values have not already been queried, and outputs 1 if and only if

$$M \neq M' \quad \text{and} \quad \mathcal{O}_1(M) = \mathcal{O}_1(M').$$

In the real world, the oracle pair is $(H_n[F], F)$. Therefore,

$$\Pr_F [D^{A, H_n[F], F} = 1] = \Pr_F [A^{H_n[F], F} \Rightarrow (M, M') : M \neq M' \wedge H_n[F](M) = H_n[F](M')].$$

Hence,

$$\Pr_F[D^{A, H_n[F], F} = 1] = \text{Adv}_{H_n}^{\text{coll}}(A).$$

In the ideal world, the oracle pair is $(RO, \mathcal{S}^{RO})$. Thus D^A outputs 1 only if A produces two distinct messages $M \neq M'$ such that

$$RO(M) = RO(M').$$

Since RO is a uniformly random function with n -bit output, the probability that any two distinct inputs among at most $q_H + 2$ hash queries collide is bounded by the birthday bound:

$$\Pr_{RO}[D^{A, RO, \mathcal{S}^{RO}} = 1] \leq \binom{q_H + 2}{2} \cdot 2^{-n} = \frac{(q_H + 2)(q_H + 1)}{2^{n+1}}.$$

By the assumed indistinguishability of $H_n[F]$ from RO , we have

$$\left| \Pr_F[D^{A, H_n[F], F} = 1] - \Pr_{RO}[D^{A, RO, \mathcal{S}^{RO}} = 1] \right| \leq \varepsilon_{\text{ind}}.$$

Combining the above inequalities gives

$$\text{Adv}_{H_n}^{\text{coll}}(A) \leq \varepsilon_{\text{ind}} + \frac{(q_H + 2)(q_H + 1)}{2^{n+1}},$$

which proves the claim. $\square$

5.3.2 Preimage Resistance

The proof in this subsection follows the proof routine for Sponge and related constructions [6, 10]. The argument is adapted from a permutation interface to the present ideal PRNG oracle $F : \{0, 1\}^{v+k} \rightarrow \{0, 1\}^{v+n}$.

Theorem 3 (Classical Preimage Resistance). *Assume $n \leq v + k$ and $q < 2^{v+n-1}$. Let A be a classical adversary making at most q distinct oracle queries to the underlying component. Then*

$$\text{Adv}_{H_n}^{\text{pre}}(A) \leq \frac{2q}{2^n}.$$

Proof. Fix an arbitrary target value

$$h \in \{0, 1\}^n.$$

The ideal underlying component is a uniformly sampled function

$$F : \{0, 1\}^{v+k} \rightarrow \{0, 1\}^{v+n}.$$

An adversary's oracle queries to F are summarized by a transcript

$$Q = ((X_1, Y_1), \dots, (X_q, Y_q)),$$

where $X_i \in \{0, 1\}^{v+k}$, $Y_i = F(X_i) \in \{0, 1\}^{v+n}$, and repeated queries are answered consistently. Let Q_i denote the transcript up to and including the i -th fresh query.

Write every oracle output as

$$Y = Y^V \parallel Y^R, \quad |Y^V| = v, \quad |Y^R| = n,$$

where $\text{Link}(Y) = Y^V$ and $\text{HashOut}(Y) = Y^R$. These two maps are applied to the exposed PRNG output stream segment, not to a hidden internal register state. For the fixed target h , define the set of final outputs that would produce h :

$$Z_h = \{Y \in \{0, 1\}^{v+n} : \text{HashOut}(Y) = h\}.$$

Since the v -bit chaining component is unrestricted,

$$|Z_h| = 2^v.$$

Without loss of generality, before outputting a candidate preimage $M = M_0 \parallel \dots \parallel M_{\ell-1}$, the adversary has queried the underlying component along the entire rooted path needed to evaluate it;

otherwise, a wrapper can make these missing component queries and then output the same M , with the added queries counted in q . Define $\text{PRE}(Q)$ as the event that Q contains a rooted path

$$(V_0 \parallel M_0) \mapsto Y_0, \quad (\text{Link}(Y_0) \parallel M_1) \mapsto Y_1, \quad \dots, \quad (\text{Link}(Y_{\ell-2}) \parallel M_{\ell-1}) \mapsto Y_{\ell-1}$$

such that

$$Y_{\ell-1} \in Z_h.$$

If A outputs a preimage of h , then $\text{PRE}(Q)$ occurs.

Define the bad event

$$\text{BAD}(Q) : \exists i \in \{1, \dots, q\} \text{ such that } Y_i \in Z_h.$$

By the definition of $\text{PRE}(Q)$, any successful rooted preimage path has its last oracle output in Z_h . Hence

$$\text{PRE}(Q) \Rightarrow \text{BAD}(Q), \quad \Pr[\text{PRE}(Q)] \leq \Pr[\text{BAD}(Q)].$$

It remains to bound $\Pr[\text{BAD}(Q)]$. Conditioned on no previous bad event, the answer to the i -th fresh query is uniformly distributed over $\{0, 1\}^{v+n}$. Using the conservative transcript bound from the random-function analysis,

$$\Pr[\text{BAD}(Q_i) \mid \neg \text{BAD}(Q_{i-1})] \leq \frac{|Z_h|}{2^{v+n} - (i-1)} = \frac{2^v}{2^{v+n} - (i-1)}.$$

Since $q < 2^{v+n-1}$, the denominator is at least 2^{v+n-1} , and therefore

$$\Pr[\text{BAD}(Q)] \leq \sum_{i=1}^q \frac{2^v}{2^{v+n} - (i-1)} \leq \frac{2q \cdot 2^v}{2^{v+n}} = \frac{2q}{2^n}.$$

The target h was arbitrary, so the same bound holds for the everywhere preimage advantage in the ideal-component game. $\square$

5.3.3 Second-Preimage Resistance

The second-preimage proof uses the same path-connection decomposition as the stronger second-preimage analyses for sponge-like and permutation-based hash constructions [6, 10]. Only the interface is changed: all transcript entries below are queries to the ideal PRNG oracle rather than to a public permutation.

Theorem 4 (Classical Second-Preimage Resistance). *For a fixed challenge message M , let*

$$M_0 \parallel \dots \parallel M_{\alpha-1} = \text{Pad}(M)$$

be its padded block sequence. Let A make at most q distinct oracle queries to the underlying component, and let

$$\bar{q} = q + \alpha.$$

Assume $\bar{q} < 2^{v+n-1}$. Then

$$\text{Adv}_{H_n}^{M-2\text{pre}}(A) \leq \frac{2\bar{q}}{2^n} + \frac{2\alpha\bar{q}}{2^v}.$$

Proof. Let

$$h = H_n[F](M).$$

As in the preimage proof, the adversary's oracle queries to the ideal component are represented by a transcript

$$Q = ((X_1, Y_1), \dots, (X_{\bar{q}}, Y_{\bar{q}})),$$

where $Y_i = F(X_i)$. The transcript includes the α oracle queries needed to evaluate the known first preimage M ; these queries are given to the adversary for free before it starts making its own q queries. Let Q_i be the transcript up to the i -th fresh pair, and assume without loss of generality that no query is repeated.

Denote the known rooted path for M by

$$\bar{V}_0 = V_0, \quad \bar{X}_j = \bar{V}_j \parallel M_j, \quad \bar{Y}_j = F(\bar{X}_j), \quad \bar{V}_{j+1} = \text{Link}(\bar{Y}_j),$$

for $0 \leq j \leq \alpha - 1$. Then

$$\text{HashOut}(\bar{Y}_{\alpha-1}) = h.$$

Define the target-output set

$$Z_h = \{Y \in \{0, 1\}^{v+n} : \text{HashOut}(Y) = h\}, \quad |Z_h| = 2^v.$$

A successful second preimage $M' \neq M$ induces a different rooted path in the transcript whose final output belongs to Z_h . Let $\text{SEC}(Q)$ be the event that Q contains such a rooted path:

$$(V_0 \parallel M'_0) \mapsto Y'_0, \quad (\text{Link}(Y'_0) \parallel M'_1) \mapsto Y'_1, \quad \dots, \quad (\text{Link}(Y'_{\beta-2}) \parallel M'_{\beta-1}) \mapsto Y'_{\beta-1},$$

with

$$\text{Pad}(M') = M'_0 \parallel \dots \parallel M'_{\beta-1} \neq \text{Pad}(M), \quad Y'_{\beta-1} \in Z_h.$$

If A outputs a valid second preimage, then $\text{SEC}(Q)$ occurs.

As in the SPONGE-DM second-preimage proof framework [10], the adversary can succeed in one of two ways. It may create a new final output in Z_h , or it may connect an absorption path to the already known first-preimage path. We capture these cases by two bad events. First,

$$\text{BAD}(Q) : \exists (X, Y) \in Q \text{ such that } Y \in Z_h \text{ and } (X, Y) \neq (\bar{X}_{\alpha-1}, \bar{Y}_{\alpha-1}).$$

The exclusion removes the trivial final query of the known first preimage. Second, let

$$\mathcal{C}_M = \{\bar{V}_0, \bar{V}_1, \dots, \bar{V}_{\alpha-1}\}$$

be the set of chaining values that occur as inputs along the known first-preimage path. Define

$$\text{CONNECT}(Q) : \exists (X, Y) \in Q \text{ such that } \text{Link}(Y) \in \mathcal{C}_M \text{ and } (X, Y) \neq (\bar{X}_j, \bar{Y}_j) \text{ for every } j.$$

If a second-preimage path reaches the known final edge without creating a new output in Z_h , then it must have joined the known path at one of the chaining values in $\mathcal{C}_M$. Hence

$$\text{SEC}(Q) \Rightarrow \text{BAD}(Q) \vee \text{CONNECT}(Q),$$

and therefore

$$\Pr[\text{SEC}(Q)] \leq \Pr[\text{BAD}(Q)] + \Pr[\text{CONNECT}(Q)].$$

We bound the two probabilities separately. The BAD analysis is the same as in the preimage proof, except that the transcript has length $\bar{q}$ and the known final edge is excluded:

$$\Pr[\text{BAD}(Q)] \leq \sum_{i=1}^{\bar{q}} \frac{|Z_h|}{2^{v+n} - (i-1)} \leq \frac{2\bar{q}}{2^n}.$$

For CONNECT, the set $\mathcal{C}_M$ has size at most α . For any fixed chaining value V , there are 2^n outputs Y with $\text{Link}(Y) = V$. Thus the number of outputs that connect to $\mathcal{C}_M$ is at most $\alpha 2^n$, and

$$\Pr[\text{CONNECT}(Q)] \leq \sum_{i=1}^{\bar{q}} \frac{\alpha 2^n}{2^{v+n} - (i-1)} \leq \frac{2\alpha\bar{q}}{2^v}.$$

The assumption $\bar{q} < 2^{v+n-1}$ is used in both sums to lower bound the denominator by 2^{v+n-1} . Combining the bounds gives

$$\Pr[\text{SEC}(Q)] \leq \frac{2\bar{q}}{2^n} + \frac{2\alpha\bar{q}}{2^v},$$

which is the claimed pointwise second-preimage bound for the fixed challenge message M . $\square$

5.4 Quantum Security

Since the proof-level primitive is the ideal PRNG

$$F : \{0, 1\}^{v+k} \rightarrow \{0, 1\}^{v+n},$$

namely a random function rather than an invertible permutation or ideal cipher, the appropriate quantum proof tool is the QROM lifting theorem for search-type games from Yamakawa and Zhandry [11]. Concretely, we model F as a quantum-accessible random oracle with domain $\{0, 1\}^{v+k}$ and range $\{0, 1\}^{v+n}$, and we reduce each quantum security experiment to a search-type relation game whose winning condition can be verified from a bounded number of classical oracle input-output pairs. We then apply the ROM-to-QROM lifting theorem for search-type games. This is in line with the quantum security proofs for sponge-based hash constructions [5].

Quantum adversaries and advantage notions. Following the standard QROM convention, each classical advantage notion has a quantum analogue obtained by allowing the adversary to issue superposition queries to the underlying ideal PRNG oracle F . The output of the adversary remains classical. Thus a quantum collision adversary A_Q has advantage

$$\text{Adv}_{H_n}^{q\text{-coll}}(A_Q) = \Pr_{F \leftarrow \mathfrak{F}_{v,k,n}} [A_Q^F \Rightarrow (M, M') : M \neq M' \wedge H_n[F](M) = H_n[F](M')],$$

where A_Q^F denotes quantum oracle access to the sampled random function F .

For a fixed target h , the quantum h -preimage advantage is

$$\text{Adv}_{H_n}^{q\text{-}h\text{-pre}}(A_Q) = \Pr_{F \leftarrow \mathfrak{F}_{v,k,n}} [A_Q^F(h) \Rightarrow M : H_n[F](M) = h],$$

and the quantum everywhere-preimage advantage is

$$\text{Adv}_{H_n}^{q\text{-epre}}(A_Q) = \max_{h \in \{0,1\}^n} \text{Adv}_{H_n}^{q\text{-}h\text{-pre}}(A_Q).$$

For a fixed challenge message M , the quantum second-preimage advantage is

$$\text{Adv}_{H_n}^{q\text{-}M\text{-spr}}(A_Q) = \Pr_{F \leftarrow \mathfrak{F}_{v,k,n}} [A_Q^F(M) \Rightarrow M' : M' \neq M \wedge H_n[F](M') = H_n[F](M)].$$

For bl -bit challenge messages, the quantum everywhere second-preimage advantage is

$$\text{Adv}_{H_n}^{q\text{-espr}[bl]}(A_Q) = \max_{M \in \{0,1\}^{bl}} \text{Adv}_{H_n}^{q\text{-}M\text{-spr}}(A_Q).$$

5.4.1 QROM Lifting Lemma Used in the Proof

We use the lifting theorem for search-type games in the QROM from Yamakawa and Zhandry [11, Theorem 6 and Theorem 7]. Specialized to the present random-function setting, the lifting theorem implies the following relation-search form.

Lemma 1 (QROM lifting for bounded-query relation games). *Let R be a search relation whose winning condition is determined by ρ oracle input-output pairs of the random function F , where ρ is a constant independent of the adversary. Let A_Q be a quantum adversary making at most q quantum queries to F . Then there exists a classical adversary B making at most ρ classical queries to F such that*

$$\Pr[A_Q^F \text{ wins } R] \leq \Gamma_\rho(q) \Pr[B^F \text{ wins } R], \quad \Gamma_\rho(q) := (2q + 1)^{2\rho}.$$

In the sequel, the only values of ρ that arise are $\rho = 1$ and $\rho = 2$. Thus the quantum bounds are obtained by identifying a one-query or two-query search relation and multiplying the corresponding classical one- or two-query success probability by $\Gamma_1(q)$ or $\Gamma_2(q)$, respectively.

5.4.2 Quantum Collision Resistance

Theorem 5 (Quantum Collision Resistance). *Let A_Q make at most q quantum queries to the underlying ideal PRNG oracle F . Then, in the QROM for the ideal PRNG oracle F ,*

$$\text{Adv}_{H_n}^{q\text{-coll}}(A_Q) \leq \Gamma_2(q) \left(\varepsilon_{\text{ind}} + \frac{1}{2^n} \right).$$

Here ε_{ind} is the indistinguishability error for the constant-cost classical verifier appearing in Section 5.3.1; by Theorem 1, this term is $O(2^{-v})$.

Proof. The classical collision argument in Section 5.3.1 already gives the relevant classical advantage bound. Namely, any classical collision verifier for two candidate messages uses two classical evaluations to check the final equality, and its success probability is bounded by the random-oracle collision probability plus the indistinguishability error:

$$\text{Adv}_{H_n}^{\text{coll}}(B_{\text{coll}}) \leq \varepsilon_{\text{ind}} + 2^{-n}.$$

The collision-winning condition is determined by two classical input-output pairs, so $\rho = 2$ in Lemma 1. Therefore, for the quantum adversary A_Q , the lifting lemma gives a two-query classical adversary B_{coll} such that

$$\text{Adv}_{H_n}^{q\text{-coll}}(A_Q) \leq \Gamma_2(q) \text{Adv}_{H_n}^{\text{coll}}(B_{\text{coll}}).$$

Substituting the classical collision bound from Section 5.3.1 yields

$$\text{Adv}_{H_n}^{q\text{-coll}}(A_Q) \leq \Gamma_2(q)(\varepsilon_{\text{ind}} + 2^{-n}).$$

Finally, Theorem 1 gives $\varepsilon_{\text{ind}} = O(2^{-v})$ for this constant-cost verifier, so the collision bound has the asymptotic form $\Gamma_2(q)(2^{-n} + 2^{-v})$. $\square$

5.4.3 Quantum Preimage Resistance

Theorem 6 (Quantum Preimage Resistance). *Let A_Q make at most q quantum queries to the underlying ideal PRNG oracle F . Then, in the QROM for the ideal PRNG oracle F ,*

$$\text{Adv}_{H_n}^{q\text{-epre}}(A_Q) \leq \Gamma_1(q) \frac{2}{2^n}.$$

Proof. Fix a target value $h \in \{0, 1\}^n$. In Section 5.3.2, the classical preimage proof shows that a q_c -query classical adversary B has

$$\text{Adv}_{H_n}^{\text{pre}}(B) \leq \frac{2q_c}{2^n}.$$

The preimage-winning condition is a one-query search relation at the level of the final component output, so $\rho = 1$ in Lemma 1. Applying the lifting lemma to A_Q gives a one-query classical adversary B_{pre} such that

$$\text{Adv}_{H_n}^{q\text{-}h\text{-pre}}(A_Q) \leq \Gamma_1(q) \text{Adv}_{H_n}^{\text{pre}}(B_{\text{pre}}).$$

Now apply the classical preimage bound from Section 5.3.2 with $q_c = 1$:

$$\text{Adv}_{H_n}^{\text{pre}}(B_{\text{pre}}) \leq \frac{2}{2^n}.$$

Thus

$$\text{Adv}_{H_n}^{q\text{-}h\text{-pre}}(A_Q) \leq \Gamma_1(q) \frac{2}{2^n}.$$

The target h was arbitrary, so taking the maximum over h gives the everywhere preimage bound. $\square$

5.4.4 Quantum Second-Preimage Resistance

Theorem 7 (Quantum Second-Preimage Resistance). *For a fixed challenge message M , let*

$$M_0 \parallel \cdots \parallel M_{\alpha-1} = \text{Pad}(M)$$

be its padded block sequence. Let A_Q make at most q quantum queries to the underlying ideal PRNG oracle F . Then, in the QROM for the ideal PRNG oracle F ,

$$\text{Adv}_{H_n}^{q\text{-}M\text{-spr}}(A_Q) \leq \Gamma_1(q) \left(\frac{1}{2^n} + \frac{\alpha}{2^v} \right).$$

Consequently, if every bl-bit challenge message has padded block length at most α_{bl} , then

$$\text{Adv}_{H_n}^{q\text{-espr}[bl]}(A_Q) \leq \Gamma_1(q) \left(\frac{1}{2^n} + \frac{\alpha_{bl}}{2^v} \right).$$

Proof. The classical second-preimage proof in Section 5.3.3 gives the exact decomposition needed here. For the fixed first preimage M , a successful second-preimage adversary must trigger one of the two classical events used there:

BAD or CONNECT.

The event BAD corresponds to creating a new final output in the target set Z_h , while CONNECT corresponds to connecting to one of the α chaining values on the known rooted path of M . The per-query classical bounds proved in Section 5.3.3 are

$$\Pr[\text{BAD}] \leq 2^{-n}, \quad \Pr[\text{CONNECT}] \leq \alpha 2^{-v}.$$

Hence a one-query classical second-preimage adversary B_{spr} , after the rooted path of M is fixed, satisfies

$$\text{Adv}_{H_n}^{M\text{-}2\text{pre}}(B_{\text{spr}}) \leq 2^{-n} + \alpha 2^{-v}.$$

The second-preimage winning condition is therefore a one-query search relation once the challenge path of M is fixed. Applying Lemma 1 with $\rho = 1$, there exists such a one-query classical adversary B_{spr} with

$$\text{Adv}_{H_n}^{q-M-\text{spr}}(A_Q) \leq \Gamma_1(q) \text{Adv}_{H_n}^{M-2\text{pre}}(B_{\text{spr}}).$$

Substituting the classical bound from Section 5.3.3 yields

$$\text{Adv}_{H_n}^{q-M-\text{spr}}(A_Q) \leq \Gamma_1(q)(2^{-n} + \alpha 2^{-v}).$$

Maximizing over all bl -bit challenge messages and replacing α by the maximal padded block length α_{bl} gives the everywhere second-preimage bound. $\square$

For the table below, α denotes the padded block length of the first preimage in the second-preimage experiment. The quantum collision entry is the bound obtained from the lifting theorem; it is conservative and is not claimed to be tight.

Table 3: Generic proof bounds for an n -bit NEULASER output		
Security notion	Classical generic cost	Quantum proof cost
Collision	$\min\{2^{n/2}, 2^{v/2}\}$	$\min\{2^{n/4}, 2^{v/4}\}$
Preimage	2^n	$2^{n/2}$
Second preimage	$\min\{2^n, 2^v/\alpha\}$	$\min\{2^{n/2}, 2^{v/2}/\sqrt{\alpha}\}$

6 Other security features

6.1 Cryptanalysis of the underlying PRNG

The underlying PRNG follows the same broad design philosophy as ZUC [1]: a long-period word-oriented register structure provides the state backbone, while a compact nonlinear word function combines modular addition, XOR, rotation-based diffusion, and parallel 8-bit S-boxes to destroy linear relations before output is extracted. The main differences are that NEULASER uses $\mu \in \{3, 4, 5\}$ coupled 16-stage NLSR modules over the 32-bit prime field $\mathbb{F}_p$, and that the primitive is used as a bounded-output PRNG inside a hash construction rather than as a conventional stream cipher.

Linear backbone and period. For each module, the linear core is defined over $p = 2^{32} - 5$, and its characteristic polynomial $\chi_{\text{lin}}(x)$ is primitive over $\mathbb{F}_p$. Thus, before nonlinear feedback and inter-module coupling are added, each nonzero module has maximum linear period $p^{16} - 1$. This property is analogous to the role of the long LFSR in ZUC: it prevents short purely linear cycles from being built into the register backbone. In the full PRNG, the nonlinear local feedback and the global confusion layer perturb this maximum-period backbone every round, so attacks must handle both the prime-field recurrence and the word-level nonlinear transformations.

Resistance to linear approximations. A purely linear attack would need useful correlations through the local feedback and the global confusion function. The function F prevents this in several ways. First, its two input branches mix modular addition and XOR, so linear masks over $\mathbb{F}_2$ do not pass through the arithmetic part without carry-dependent disturbance. Second, the ZUC-style layers $\mathcal{L}_1$ and $\mathcal{L}_2$ spread each 32-bit word before the S-box layer; when the input and output are viewed as four bytes, their linear and differential branch numbers are both 5. Third, the fixed inversion-affine S-box has maximum Walsh coefficient 32, or equivalently component-function nonlinearity 112, which is optimal for 8-bit bijective S-boxes. Consequently, even low-weight masks entering F are forced through several active byte S-boxes after a small number of rounds. The global confusion layer reuses F on words selected from different modules, so a linear approximation must also remain consistent across the round-dependent inter-module wiring.

Resistance to differential attacks. The differential behavior is controlled by the same two-level diffusion mechanism. The local layer uses endpoint and middle words $(s_{15}, s_{10}, s_5, s_0)$, so a difference affecting the shift register is repeatedly injected into F at separated tap positions. The rotation-xor layers activate multiple S-box bytes, and the inversion-affine S-box has the standard optimal differential

uniformity 4 for an 8-bit inversion-based bijection. In addition, the modular additions in F and in the final reduction modulo p introduce carry propagation, making bitwise differential trails dependent on arithmetic carries rather than on a purely XOR-linear model. The global confusion function then injects Γ_i into several positions of each module. As a result, a local difference cannot remain confined to one register module: after the global layer, it affects other modules and returns to the local feedback in later rounds.

Algebraic and state-recovery attacks. A direct state-recovery attack must reconstruct 512μ bits of internal state, namely 1536, 2048, or 2560 bits for the three variants. Only $v + n$ output bits are relevant in one hash-mode invocation, and the hash construction uses only v bits for ordinary chaining blocks and an additional n bits on the final block. Thus an attacker does not receive an unbounded keystream from one initialized state. Algebraically, each update round contains inversion-affine byte S-boxes, modular additions with carries, reduction modulo p , and round-dependent module selection. These operations rapidly raise the degree of equations describing the output in terms of the initialized state. Compared with a single-register guess-and-determine strategy, the coupled design forces guessed words in one module to be checked against globally mixed words in the other modules.

Initialization and output separation. Every hash block initializes a fresh PRNG invocation from the pair (V_i, M_i) , where M_i has k bits and V_i has v bits. The block constants C_i are xored into the chaining side, so identical message blocks at different positions do not induce the same public chaining input. Moreover, the construction uses disjoint output portions: the v -bit prefix is used for the next chaining value, while the final digest is taken from the following n -bit portion only on the last block. This separation is important for the hash setting, because it prevents the public digest from revealing the continuation state.

Overall, the current cryptanalytic assessment does not identify a shortcut that avoids recovering a large coupled state or exploiting a structural weakness in F . The PRNG inherits several conservative design ideas from ZUC—a long word-oriented register backbone, nonlinear word filtering, rotation-xor diffusion, and inversion-based byte substitution—while adding multiple coupled modules and bounded-output use inside the hash mode. This analysis is not a substitute for the ideal-PRNG proofs of Section 5; rather, it explains why the proposed concrete PRNG is intended to approximate the ideal component used in those proofs.

6.2 Statistical Randomness Tests

The statistical behavior of the produced digest streams was evaluated using the methodology of the NIST SP 800-22 statistical test suite for random and pseudorandom generators [9]. These tests are not a substitute for cryptanalysis, but they are useful sanity checks for detecting obvious biases or short-range statistical defects in the output distribution.

For each of the three variants, 100 binary sequences were generated, each of length 10^6 bits. The j -th sequence was obtained by hashing the structured 128-bit counter messages

$$\text{enc}_{64}(j) \parallel \text{enc}_{64}(c), \quad c = 0, 1, 2, \dots,$$

where enc_{64} denotes big-endian 64-bit encoding. Digest bits were concatenated until 10^6 bits were obtained and then truncated to the required sequence length. The tests were run on the reference implementation.

The significance level was set to $\alpha = 0.01$. For each test, an individual sequence passes when its p -value is at least α . Across the 100 sequences, the required pass count is computed using the NIST 3σ proportion rule,

$$100 \left(1 - \alpha - 3\sqrt{\frac{\alpha(1-\alpha)}{100}} \right),$$

which gives a required pass count of 97. In addition, the 100 p -values for each test were divided into ten equal subintervals of $[0, 1]$, and a chi-square uniformity test was applied; the uniformity criterion is $p_{\text{uniform}} \geq 10^{-4}$.

The implemented test set consists of the monobit frequency test, block frequency test with block length 10000, runs test, longest run of ones in a block with block length 10000, cumulative sums test in both forward and reverse directions, serial test with pattern length $m = 16$ (two p -value series), and approximate entropy test with $m = 10$.

Table 4: Statistical randomness test results for NEULASER digest streams. Each cell gives pass count / uniformity p -value.

Test	NEULASER-512	NEULASER-768	NEULASER-1024
Frequency	100/0.366918	99/0.935716	98/0.401199
Block frequency	100/0.834308	99/0.719747	100/0.096578
Runs	100/0.334538	100/0.023545	99/0.419021
Longest run	98/0.955835	99/0.955835	100/0.401199
Cusum forward	100/0.181557	100/0.798139	98/0.883171
Cusum reverse	99/0.171867	99/0.719747	98/0.534146
Serial P_1	99/0.759756	100/0.249284	98/0.955835
Serial P_2	100/0.971699	98/0.437274	98/0.249284
Approximate entropy	97/0.181557	99/0.554420	99/0.275709

All tests in Table 4 satisfy both acceptance conditions: the pass count is at least 97, and the uniformity p -value is greater than 10^{-4} . Thus, under this NIST-style statistical test configuration, no detectable output bias was observed for NEULASER-512, NEULASER-768, or NEULASER-1024.

6.3 Resistance to Length Extension Attacks

A length extension attack attempts to compute the hash of an extended message from the digest of a shorter message, without reprocessing the shorter message. This is the classical weakness of Merkle–Damgård hashes whose public digest is exactly the final chaining value. In such a construction, once an adversary knows $H(M)$ and the length of M , it can often use $H(M)$ as the initial chaining state and continue the compression function on additional blocks.

NEULASER does not expose such a continuation state. Let

$$M^* = M_0 \parallel M_1 \parallel \cdots \parallel M_{\ell-1}$$

be the padded form of M . The last PRNG invocation produces

$$Z_{\ell-1} = Z_{\ell-1}^V \parallel Z_{\ell-1}^R \in \{0, 1\}^{v+n},$$

where the digest is

$$H_n(M) = Z_{\ell-1}^R.$$

If the computation were to be continued after the padded prefix M^* , the next chaining value would have to be

$$V_\ell = Z_{\ell-1}^V \oplus C_\ell.$$

This value is not part of the public digest. Thus the value needed for continuation is the hidden v -bit slice $Z_{\ell-1}^V$, while the public digest is the disjoint n -bit slice $Z_{\ell-1}^R$.

In the ideal-PRNG model, for a fresh last-block input $(V_{\ell-1}, M_{\ell-1})$, the two output components $Z_{\ell-1}^V$ and $Z_{\ell-1}^R$ are uniform and independent. Conditioned on the known digest value $H_n(M) = Z_{\ell-1}^R$, the hidden chaining component $Z_{\ell-1}^V$ remains uniformly distributed over $\{0, 1\}^v$. Therefore an adversary that only knows $H_n(M)$ cannot derive the correct continuation value except by guessing a v -bit state. With q independent continuation attempts, the generic success probability is at most $q/2^v$, before considering the ordinary chance of finding a collision or second preimage for the whole hash function.

7 Performance Evaluation

The dominant primitive operation is the nonlinear word function F . Each call to $F(R_0, R_1, R_2, R_3)$ contains one modular addition modulo 2^{32} , one word XOR, one 64-bit rotation by 16 bits, two ZUC-style rotation-xor linear layers $\mathcal{L}_1, \mathcal{L}_2$, eight parallel 8-bit S-box lookups, one 32-bit rotation by 8 bits, one XOR, and one final modular addition modulo 2^{32} . These operations map directly to common integer instructions on mainstream x86 processors and do not require large tables or external cryptographic libraries.

The μ -module structure is also favorable for software performance. Local feedback values can be computed independently before the global confusion words are formed, and the same compact word function

F is reused in both the local and global layers. The current reference implementation is written for clarity and portability in C99; it does not use hand-written assembly, SIMD intrinsics, or multi-threading. Further performance-oriented versions may exploit the module-level parallelism with vectorization or thread-level parallelism.

7.1 Benchmark Methodology

The software performance test follows the input sizes recommended in the x86 self-assessment guideline. The eight message lengths are 32, 128, 512, 1024, 4096, 8192, 16384, and 65536 bytes, denoted by S1–S8. For each message length and each NEULASER variant, the benchmark performs 100 measurements. Each measurement repeats the `CryptHash` call enough times to process 65536 input bytes, so every table entry is averaged over 6553600 processed input bytes. The input buffer is generated deterministically by a fixed xorshift rule. The benchmark pins the process to one CPU core and reports both wall-clock throughput and cycle counts measured with the x86 time-stamp counter.

The reference implementation was compiled with

```
gcc -std=c99 -Wpedantic -Wall -Wextra -O2.
```

The benchmark source is `benchmark_hash.c` in the reference implementation directory. The measured results are summarized directly in Table 6.

Table 5: Software implementation environment.

Item	Configuration
Processor	Intel Core i7-1165G7, nominal 2.80 GHz
Cores/threads used	1 core used for testing; processor has 4 cores / 8 threads
Memory	16478896 KiB physical memory
Operating system	Windows NT 10.0.26200, 64-bit, MSYS2 runtime 3.6.9
Instruction set	x86-64 with SSE2, AVX, AVX2, AES-NI, SHA-NI support
Compiler	GCC 15.2.0, target <code>x86_64-w64-mingw32</code>
Build tool	GNU Make 4.4.1
Implementation language	ISO C99 reference implementation
External cryptographic libraries	None
Test time	2026-06-29, Asia/Shanghai time zone

Table 6: Reference implementation hash throughput and cycle cost.

Level	Bytes	NEULASER-512		NEULASER-768		NEULASER-1024	
		MB/s	cyc/B	MB/s	cyc/B	MB/s	cyc/B
S1	32	6.26	426.8959	4.82	555.0435	3.21	831.7329
S2	128	13.66	195.7655	19.58	136.5581	15.06	177.5529
S3	512	23.17	115.3915	21.85	122.3523	22.69	117.8054
S4	1024	26.06	102.6004	25.56	104.5745	22.73	117.6302
S5	4096	27.07	98.7637	23.99	111.4357	25.23	105.9710
S6	8192	27.29	97.9577	20.71	129.0785	25.38	105.3248
S7	16384	27.70	96.5241	26.17	102.1446	25.05	106.7167
S8	65536	27.79	96.2107	26.39	101.2941	25.22	106.0057

7.2 Bottleneck Analysis

To identify the dominant cost, an additional function-level profiling run was performed on the reference implementation. The profiling build used GCC function instrumentation with inlining disabled, so the absolute timing numbers are not used as throughput measurements. The resulting percentages are used only to locate the computational bottleneck. The input was the S8 case, namely 100 independent calls on 65536-byte messages.

For a 65536-byte message, the number of padded message blocks is large because the block size is k bits. Each block invokes the PRNG once, and each PRNG invocation performs 32 blank rounds before producing output. Table 7 gives the resulting operation counts per message.

The profiling result is summarized in Table 8. Almost all time is spent inside the PRNG, and in particular inside the state-update function. Message padding, block construction, IV generation, and

Table 7: Main operation counts for one 65536-byte message.

Variant	Padded blocks	Stream rounds/block	Final rounds	Update calls	F calls
NEULASER-512	547	6	12	20792	124752
NEULASER-768	432	7	13	16854	134832
NEULASER-1024	357	7	14	13930	139300

digest copying together account for less than 0.1% of the running time in this test. The entries “linear core” and “ F function” are subcosts inside **Update**, so they should not be added to the **Update** row.

Table 8: Function-level bottleneck profile for the reference implementation.

Component	NEULASER-512	NEULASER-768	NEULASER-1024
PRNG invocations, inclusive	99.86%	99.90%	99.92%
Update , inclusive	98.33%	98.48%	98.56%
Linear core, inclusive	2.52%	2.52%	2.52%
F function, inclusive	44.98%	45.10%	45.25%
Block construction	0.06%	0.04%	0.04%

The main reason for the observed cost is therefore structural rather than input/output overhead. The hash mode reinitializes a large-state PRNG for each padded block. Every invocation pays a fixed cost of 32 blank rounds, and every update round evaluates 2μ copies of the nonlinear word function F . The linear core is executed once per module per update, but its measured contribution is small compared with the nonlinear F function. Hence the dominant cost comes from repeatedly running the PRNG update schedule, especially the nonlinear feedback and global-confusion computations based on F . Future optimized implementations should target this bottleneck while keeping the submitted `CryptHash` interface and test-vector behavior unchanged.

7.3 Future Optimization Directions

The current submission reports the portable C99 reference implementation. Several implementation-level optimizations are natural candidates for future work.

First, the reduction modulo $p = 2^{32} - 5$ can be specialized. For a 64-bit value $x = x_0 + 2^{32}x_1$, one may use the congruence $2^{32} \equiv 5 \pmod{p}$ and fold x to $x_0 + 5x_1$, followed by a small number of conditional subtractions. This avoids general integer division in the linear core and in any other arithmetic modulo p .

Second, the coupled-NLSR update exposes module-level parallelism. The local feedback values of the μ modules can be computed independently before the global-confusion layer is applied. A production implementation can schedule these independent computations to improve instruction-level parallelism and reduce pipeline stalls.

Third, the register-shift part of the NLSR update is a regular word movement pattern. It is therefore a candidate for SIMD-assisted loads, stores, and shuffles on platforms with SSE2, AVX2, or later vector instruction sets. The benefit should be evaluated carefully because the nonlinear F calls still form a large part of the update cost.

Fourth, message-level parallelism can be exploited when many independent messages are hashed. A batch API could distribute independent `CryptHash` computations across multiple cores or SIMD lanes without changing the standard single-message API. Such a batch interface would be an implementation extension rather than a change to the hash function definition.

8 Feature Statement

The following statements summarize notable properties of the NEULASER design.

1. **Innovation.** NEULASER departs from classical Merkle–Damgård and sponge paradigms by using a coupled-NLSR PRNG inside a chain-based hash mode. Message blocks seed successive PRNG invocations, and chaining values are derived from generated output streams rather than from a conventional compression-function output.

2. **Simplicity.** The core primitive is constructed from a small set of well-understood operations (rotations, modular addition, XOR, a single 8-bit bijective S-box, and ZUC-style rotation-xor linear layers). Reusing the same F function for local and global mixing reduces implementation surface and eases verification.
3. **Flexibility.** The design cleanly separates primitive mechanics from hash-mode parameters such as the fixed initialization vector, chaining-value length, seed/key length, and output length. This permits tailoring the PRNG instantiation or the hash mode parameters without changing the proof structure.
4. **Extendable Output.** The construction can be naturally extended to an extendable-output function (XOF). Since the underlying component is a PRNG, the final invocation can continue producing a pseudorandom output stream after the chaining portion. For any requested output length, the hash value can be obtained by continuing the existing output procedure and truncating the generated bit stream to the required number of bits.
5. **Cross-platform Performance Advantages.** Word-oriented operations map efficiently to modern CPUs (native 32/64-bit arithmetic) and are amenable to SIMD/vectorization and multi-threading. The μ -module parallelism also enables high throughput in software and compact, low-latency hardware implementations.

9 Conclusion

NEULASER introduces a PRNG-based hash construction as its central design idea. Instead of building a conventional compression function or iterating a public permutation, the construction treats each padded message block as the seed/key input of a PRNG invocation and uses separated portions of the generated stream for chaining and final output. This gives a generalized MD-like iterative structure whose security is tied to the pseudorandomness of the underlying generator, while retaining a proof style close to sponge indistinguishability arguments.

The concrete instantiation uses a large-state coupled-NLSR PRNG. The underlying primitive reuses a compact nonlinear word function for both local feedback and global mixing, and the hash mode derives chaining and output material from disjoint output-stream components so that the public digest is not itself a continuation state.

The specification provides three digest-length variants (NEULASER-512, NEULASER-768, and NEULASER-1024) that share the same coupled-NLSR design, parameterized by μ , and a fixed public initialization vector, and differ in output length and state size. The hash mode admits an indistinguishability-style proof in the ideal PRNG model, with a birthday-type term for hitting the simulator’s saturated set.

The document also gives classical and quantum generic security bounds for collision, preimage, and second-preimage resistance, analyzes resistance to length extension attacks, and discusses the concrete PRNG against standard linear, differential, algebraic, and state-recovery considerations. A portable C99 reference implementation, KAT vectors, statistical randomness tests, and x86 performance self-assessment materials are provided to make the specification reproducible.

Future work will focus on deeper independent cryptanalysis of the concrete coupled-NLSR PRNG and on optimized implementations. Natural implementation directions include specialized reduction modulo $2^{32} - 5$, better instruction scheduling across the μ modules, SIMD-assisted state movement, and batch hashing of independent messages, while keeping the standard `CryptHash` interface and test-vector behavior unchanged.

References

- [1] 3GPP. Specification of the 3gpp confidentiality and integrity algorithms 128-eea3 and 128-eia3; document 2: ZUC specification. Technical Specification TS 35.222, 3rd Generation Partnership Project (3GPP), 2025. https://www.3gpp.org/ftp/Specs/archive/35_series/35.222/.
- [2] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. ECRYPT Hash Workshop 2007, 2007. <https://keccak.team/files/SpongeFunctions.pdf>.

- [3] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the sponge construction. In *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 181–197. Springer, 2008. https://doi.org/10.1007/978-3-540-78967-3_11.
- [4] Ivan Bjerre Damgård. A design principle for hash functions. In *Advances in Cryptology – CRYPTO '89 Proceedings*, volume 435 of *LNCS*, pages 416–427. Springer, 1990. https://doi.org/10.1007/0-387-34805-0_39.
- [5] Chun Guo, Kai Hu, Shuntian Jiang, Yanhong Fan, Yong Fu, Bart Preneel, and Meiqin Wang. Permutation-based hash from non-idealized assumptions: Adding feed-forward to sponge. Cryptology ePrint Archive, Paper 2025/1006, 2025. <https://eprint.iacr.org/2025/1006>.
- [6] Charlotte Lefevre and Bart Mennink. Tight preimage resistance of the sponge construction. In *Advances in Cryptology – CRYPTO 2022*, volume 13510 of *LNCS*, pages 185–204. Springer, 2022. https://doi.org/10.1007/978-3-031-15985-5_7.
- [7] Ralph C. Merkle. One way hash functions and DES. In *Advances in Cryptology – CRYPTO '89 Proceedings*, volume 435 of *LNCS*, pages 428–446. Springer, 1990. https://doi.org/10.1007/0-387-34805-0_40.
- [8] Phillip Rogaway and Thomas Shrimpton. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In *Fast Software Encryption*, volume 3017 of *LNCS*, pages 371–388. Springer, 2004. https://doi.org/10.1007/978-3-540-25937-4_24.
- [9] Andrew Rukhin, Juan Soto, James Nechvatal, Miles Smid, Elaine Barker, Stefan Leigh, Mark Levenson, Mark Vangel, David Banks, Alan Heckert, James Dray, and San Vo. A statistical test suite for random and pseudorandom number generators for cryptographic applications. NIST Special Publication 800-22 Revision 1a, National Institute of Standards and Technology, 2010.
- [10] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance. Cryptology ePrint Archive, Paper 2025/963, 2025. <https://eprint.iacr.org/2025/963>.
- [11] Takashi Yamakawa and Mark Zhandry. Classical vs quantum random oracles. In *Advances in Cryptology – EUROCRYPT 2021*, volume 12697 of *LNCS*, pages 568–597. Springer, 2021.